Rheinisch-Westfälische Technische Hochschule Aachen

Institut für physikalische Chemie

A machine learning model for the prediction of hessian matrices for two to four atomic molecules.

?

Research Internship Report

submitted by

Gereon Feldmann

presented to the

Mathematisch-Naturwissenschaftlichen Fakultät der Rheinisch-Westfälischen Technischen Hochschule Aachen

Aachen, April 2022

Contents

List of Figures

List of Tables

List of abbreviations

APF	Atom Pair Focused
BO	Born–Oppenheimer
CS	Coordinate System
ETR	Extra Tree Regression
ES	Electrostatic
LDA	Local Density Approximation
MI	Main Inertial
ML	Machine Learning
RFR	Random Forest Regression
XC	Exchange and Correlation

1 Aim

2 Motivation

3 Theory

For the computation of the features and target vectors for the ML models, the GFN2–xTB method is used. For a invariant system, a rotation of the features and the target is necessary. These rotations are done with Euler angles. As the target value are the hessian matrices, a short introduction to the matrices is done. At last two algorithms used for machine learning are discussed.

3.1 Euler Angles

One method of describing coordinate system (CS) transformations, are the Euler angles. They serve as a mathematical tool to connect one CS with another.
The initial coordinate system $\mathbf{x}, \mathbf{y}, \mathbf{z}$, shown in Fig. 3.1 can be transformed into the $\mathbf{x}', \mathbf{y}', \mathbf{z}'$ CS by three rotations. The three Euler angles α, β and γ are defined as follows. The distance between the vectors $\mathbf{L}_\mathrm{a}$ and $\mathbf{L}_\mathrm{b}$ is shown in red, and is the cutting line $\mathbf{L}$ between the **xy** and **x'y'** planes. Here, α is the angle between $\mathbf{x}$ and $\mathbf{L}_\mathrm{a}$ and β is the angle between the two z–axes $\mathbf{z}$ and $\mathbf{z}'$. The angle γ is the angle between $\mathbf{L}_\mathrm{a}$ and $\mathbf{x}'$.

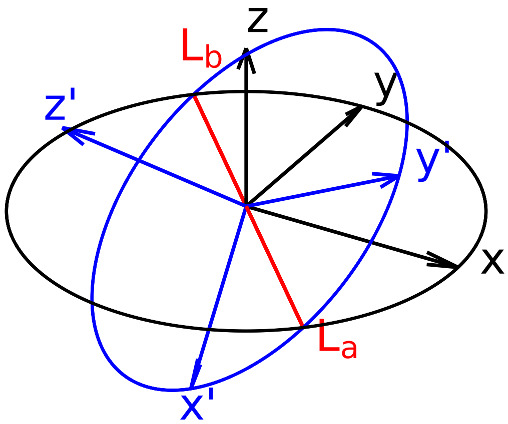

Figure 3.1: Depiction of two coordinate systems $\mathbf{x}, \mathbf{y}, \mathbf{z}$ (black) and $\mathbf{x}', \mathbf{y}', \mathbf{z}'$ (blue). The red cutting line of the **xy** and **x'y'** planes is between the vector $\mathbf{L}_\mathrm{a}$ and $\mathbf{L}_\mathrm{b}$.

For the rotation of the CS it is necessary to apply the rotation matrices in a specific order. The first rotation is around $\mathbf{z}$ by the angle α, followed by the rotation $\mathbf{x}$ by β. Then a rotation around the $\mathbf{z}$ axis is done by the angle γ. The rotation matrices $\mathbf{R}_z(\theta)$, and $\mathbf{R}_x(\theta)$ are mathematically described in Eq. (3.1.1) and Eq. (3.1.2), respectively. These are the rotation matrices for a classical rotation around the $\mathbf{z}$ and $\mathbf{x}$ axes.

$$\mathbf{R}_z(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{3.1.1}$$

$$\mathbf{R}_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix} \tag{3.1.2}$$

The full rotation of the coordinate system in dependency of the Euler angles is shown in Eq. (3.1.3).

$$\mathbf{R} = \mathbf{R}_z(\gamma) \times \mathbf{R}_x(\beta) \times \mathbf{R}_z(\alpha) \tag{3.1.3}$$

3.2 Electronic Structure Theory

In this report, properties derived from electronic structure theory are used for the training of the ML models. As GFN2–xTB is used for this purpose, a short introduction to the Hartree–Fock model and Density Functional Theory is given. The introduction starts from the non–relativistic, stationary electronic Schrödinger–equation. From this the Hartree–Fock model will be derived. This is followed by Density Functional Theory, as this builds the theoretical basis of GFN2–xTB. After that the tight–binding method will be introduced from which the semiempirical method GFN2–xTB is derived. Atomic units are used for all equations in this section.

3.2.1 Hartree–Fock Model

Molecular systems in quantum mechanics can be described by a many–body wave function eigenvalue problem. The solution of this problem is given by solving the Schrödinger equation. The separation of its time–dependency, neglecting relativistic effects and the introduction of the Born–Oppenheimer (BO) approximation results in the electronic Schrödinger equation (Eq. (3.2.1)).

$$\hat{H}_\mathrm{e}\Psi_\mathrm{e} = E_\mathrm{e}\Psi_\mathrm{e} \tag{3.2.1}$$

The solution of this eigenvalue equation leads to the electronic energy E_e. The electronic many–body–wave function Ψ_e of the system can be described by the Slater determinant (Eq. (3.2.2)). The Pauli principle introduces the condition of the antisymmetry of the wave function. This condition is fulfilled by the Slater determinant. The Slater determinant consists of k spin–orbital wave functions ϕ_i, and every wave function is permuted with the coordinates $\mathbf{r}_i$ of the N electrons. Furthermore, the wave function is normalized by the factor $(N!)^{-\frac{1}{2}}$.

$$\Psi_\mathrm{e} = \frac{1}{\sqrt{N!}} \begin{vmatrix} \phi_1(\mathbf{r}_1) & \phi_2(\mathbf{r}_1) & \dots & \phi_k(\mathbf{r}_1) \\ \phi_1(\mathbf{r}_2) & \phi_2(\mathbf{r}_2) & & \vdots \\ \vdots & & \ddots & \vdots \\ \phi_1(\mathbf{r}_N) & \dots & \dots & \phi_k(\mathbf{r}_N) \end{vmatrix} \tag{3.2.2}$$

The electronic Hamiltonian $\hat{H}_\mathrm{e}$ in the Hartree–Fock model is shown in Eq. (3.2.3) and consists of the kinetic energy of the electrons and the potential energy between electrons and nuclei as also between the electrons. Furthermore, the nuclear repulsion energy, which is approximated as a constant, is added. The kinetic energy of the nuclei is separated, due to the BO–approximation.

$$\hat{H}_\mathrm{e} = \sum_{A=1}^{M}\sum_{i=1}^{N}\left(-\frac{1}{2}\nabla_i^2 - \frac{1}{|\mathbf{R}_A - \mathbf{r}_i|}\right) + \sum_{i=1}^{N}\sum_{\substack{j=1\\ i>j}}^{N}\frac{1}{|\mathbf{r}_i - \mathbf{r}_j|} + \sum_{A=1}^{M}\sum_{\substack{B=1\\ B>A}}^{M}\frac{Z_A Z_B}{|\mathbf{R}_A - \mathbf{R}_B|} \tag{3.2.3}$$

Here, $\mathbf{R}_A$ are the coordinates of atom A, $\mathbf{r}_i$ the coordinates of electron i and Z_A the nuclear charge. While, ∇_i^2 is the second derivative in respect to every coordinate of electron i.
The electronic energy can now be described with the Slater determinant and Hamiltonian in Eq. (3.2.4). This leads to the description of the energy by the one–electron operator $\hat{h}_i$, the coulomb operator $\hat{J}_i$ and the exchange operator $\hat{K}_i$.

$$E_\mathrm{e} = \sum_{i=1}^{N} \langle \phi_i | \hat{h}_i | \phi_i \rangle + \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \Big(\langle \phi_j | \hat{J}_i | \phi_j \rangle - \langle \phi_j | \hat{K}_i | \phi_j \rangle \Big) \tag{3.2.4}$$

The Rayleigh–Ritz variational principle (s. Eq. (3.2.5)) states, that the total energy E_0 of the system is always beneath or equal to the energy corresponding to the approximated wave function. This allows the search of the exact energy by optimizing the wave function (or its parameters).

$$E_0 \leq \langle \Psi | \hat{H} | \Psi \rangle \tag{3.2.5}$$

Hence, the determination of the total energy of the system is an eigenvalue problem and can be solved by minimization. To minimize the energy, Lagrangian optimization is employed, under the condition of orthonormal ϕ_i. The inner product of the wave function $\langle \phi_i | \phi_j \rangle = \delta_{ij}$ corresponds to the Kronecker delta (Eq. (3.2.6)).

$$\langle \phi_i | \phi_j \rangle = \delta_{ij} = \begin{cases} 0 \text{ if } & i \neq j \\ 1 \text{ if } & i = j \end{cases} \tag{3.2.6}$$

The derivative of the Lagrangian functional leads to the canonical Fock equation (3.2.7), which is a solution for the minimal energy.

$$\hat{f}(\mathbf{r}_i)\phi_k(\mathbf{r}_i) = \epsilon(\mathbf{r}_i)\phi_k(\mathbf{r}_i) \tag{3.2.7}$$

$$\hat{f}(\mathbf{r}_i) = \hat{h} + \sum_{j=1}^{N} (2\hat{J}_j - \hat{K}_j) \tag{3.2.8}$$

The Fock–operator $\hat{f}(\mathbf{r}_i)$ is an effective one–electron operator. This enables the determination of the spin–orbital wave functions in an iterative manner, as the Fock–operator itself depends on the spin–orbital wave functions. The iterative search is possible due to the self–consistency of the Hartree–Fock equation. Furthermore, the general optimization can be achieved due to the Rayleigh–Ritz variational principle. Important to point out is that the Fock–operator consists of the one–electron operator, the exchange–operator and the coulomb operator, while the correlation is not included.

The spin–orbital wave functions are described via a basis set approximation. This LCAO–approach is shown in Eq. (3.2.9), where the spin–orbital wave–functions are the sum of K basis functions with an associated constant.

$$\phi_i(\mathbf{r}) = \sum_{\mu}^{K} c_\mu \chi_\mu(\mathbf{r}) \tag{3.2.9}$$

Merging the basis–set approach into Eq. (3.2.7) leads to Roothaan–Hall equation shown in Eq. (3.2.15). Here, $\mathbf{F}$ is the Fock matrix, $\mathbf{C}$ a matrix of coefficients c_μ, $\mathbf{S}$ the overlap matrix and $\boldsymbol{\epsilon}$ the diagonal eigenvalue matrix. This equation is used find a solution for the energy iteratively. Starting with an initial guess of the Fock matrix, the matrix $\mathbf{C}$ is computed, with this matrix, the density matrix $\mathbf{D}$ is calculated. The density matrix $\mathbf{D}$, is then used for the calculation of the new Fock matrix.

$$\mathbf{FC} = \mathbf{SC}\boldsymbol{\epsilon} \tag{3.2.10}$$

3.2.2 Kohn–Sham Density Functional Theory

As mentioned above, the correlation between electrons is not described in the Hartree–Fock model, due to the use of a single Slater determinant.
The correlation energy can be described by two different approaches. Instead of using one single Slater determinant, a linear combination of Slater determinants can be used. This leads to the post–Hartree–Fock methods like Full–CI or MP2. The second approach is the Density Functional Theory (DFT).
The Hohenberg–Kohn theorems build the basis for the DFT. These two theorems postulate, that the energy is a functional of the density, and that this functional follows the variational principle.
The description of the energy can therefore be done via the density of the electrons, which is defined in Eq. (3.2.11). Where, n_{occ} is the occupation number of the spin–orbital wave function.

$$\rho(\mathbf{r}) = n_{\text{occ}} \sum_{i=1}^{n} |\phi_i(\mathbf{r})|^2 \tag{3.2.11}$$

The general approach is to describe the energy in dependence of the electron density $E[\rho]$. There are different approaches for this connection. The most known approach, Kohn–Sham Density Functional Theory (KS–DFT), will be discussed. The energy in KS–DFT consists of the inexact kinetic energy, coulomb integral, the exchange correlation energy and the potential field. The kinetic energy is inexact, due to the assumption of non–interacting electrons. The energy E_{XC} consists mainly of the correlation energy and the exchange energy. In theory, the energy E_{XC}

should also include the self interaction energy and a correction to the kinetic energy.

$$E[\{\phi_i\}] = T[\{\phi_i\}] + J[\rho] + E_{\mathrm{XC}}[\rho] + \int v(\mathbf{r})\rho(\mathbf{r})\mathrm{d}\mathbf{r} \tag{3.2.12}$$

The minimization of the energy is done by a Lagrangian optimization with analogue conditions compared to Hartree–Fock. This leads to the Kohn–Sham operator and a new eigenequation, which is shown in Eq (3.2.13).

$$\hat{f}^{\mathrm{KS}}(\mathbf{r}_i)\phi_k(\mathbf{r}_i) = \epsilon(\mathbf{r}_i)\phi_k(\mathbf{r}_i) \tag{3.2.13}$$

$$\hat{f}^{\mathrm{KS}}(\mathbf{r}_i) = \hat{h}_i + 2\sum_{j=1}^{n}\hat{J}_j + \hat{v}_{\mathrm{xc}} \tag{3.2.14}$$

In comparison to the Fock operator, $\hat{f}^{\mathrm{KS}}(\mathbf{r}_i)$ includes an exchange–correlation term, instead of the exchange operator. With a basis set approach analogue to the HF model, an eigenvalue equation similar to the Roothaan–Hall equation is derived.

$$\mathbf{FC} = \mathbf{SC}\boldsymbol{\epsilon} \tag{3.2.15}$$

3.2.3 GFN2–xTB Electronic Structure Method

GFN2–xTB is a Density Functional Tight Binding (DFTB) method, that is optimized for **G**eomteries, **F**requencies and **N**oncovalent interaction energies. In these DFTB methods, the electron density ρ_0 is a linear combination of atomic electron densities $\rho_{0,A}$.

$$\rho_0 = \sum_A \rho_{0,A} \tag{3.2.16}$$

The energy is expanded by a Taylor series around the electron density ρ_0. In GFN2–xTB the expansion is done up to the third order.

$$E[\rho] = E^{(0)}[\rho_0] + E^{(1)}[\rho_0, (\delta\rho)] + E^{(2)}[\rho_0, (\delta\rho)^2] + E^{(3)}[\rho_0, (\delta\rho)^3] + \ldots \tag{3.2.17}$$

Furthermore, the energy is described with the kinetic energy $T[\rho(\mathbf{r})]$ and the external potential, due to the nuclei $V_n(\mathbf{r})$. A local density approximation (LDA) is used for the description of the exchange–correlation energy $\epsilon_{\mathrm{XC}}^{\mathrm{LDA}}$. As the LDA neglects the long range correlation effects, these are described via a non–local correlation contribution $\Phi_{\mathrm{C}}^{\mathrm{NL}}(\mathbf{r}, \mathbf{r}')$. This is in contrast to other DFTB methods like DTFB3.

$$\begin{aligned} E[\rho] = \int \rho(\mathbf{r}) \Big[& T[\rho(\mathbf{r})] + V_n(\mathbf{r}) + \epsilon_{\mathrm{XC}}^{\mathrm{LDA}}[\rho(\mathbf{r})] \\ & + \frac{1}{2}\int \left(\frac{1}{|\mathbf{r}-\mathbf{r}'|} + \Phi_{\mathrm{C}}^{\mathrm{NL}}(\mathbf{r}, \mathbf{r}')\right)\rho(\mathbf{r}')\,\mathrm{d}\mathbf{r}'\Big]\mathrm{d}\mathbf{r} + E_{nn} \end{aligned} \tag{3.2.18}$$

The terms of Eq. (3.2.17) can be divided into different energy contributions. These contributions will be briefly discussed. The zeroth order term shown in Eq. (3.2.19) consists of the repulsion energy $E^{(0)}_{\text{rep}}$ and dispersion energy $E^{(0)}_{\text{disp}}$. These are typically described by Buckingham or Lennard–Jones potentials. The dispersion term arises, due to the non–local correlation functional. Furthermore, the zeroth order term energy of the valence electrons $E^{(0)}_{\text{A,valence}}$ and core electron $E^{(0)}_{\text{A,core}}$ is summed, though the energy of the core electrons is defined as zero.

$$E[\rho_0] \approx E^{(0)}_{\text{rep}} + E^{(0)}_{\text{disp}} + \sum_A \left(E^{(0)}_{\text{A,valence}} + E^{(0)}_{\text{A,core}} \right) \tag{3.2.19}$$

The first order perturbation energy $E^{(1)}[\rho_0, (\delta\rho)]$ is approximately described by a first order dispersion energy $E^{(1)}_{\text{disp}}$, and first order valence electron energy $E^{(1)}_{\text{A,valence}}$. This perturbation allows the description of covalent bonding, as the electron density so far is described as spherical densities around the atoms, with no interaction.

$$E^{(1)}[\rho_0, (\delta\rho)] \approx E^{(1)}_{\text{disp}} + \sum_A E^{(1)}_{\text{A,valence}} \tag{3.2.20}$$

The second order energy perturbation $E^{(2)}[\rho_0, (\delta\rho)^2]$ is described by the contribution of dispersion, electrostatics and exchange correlation energies. The exchange and correlation energy $E^{(2)}_{\text{XC}}$ and electrostatic energy $E^{(2)}_{\text{ES}}$ are described in GFN2–xTB with an isotropic as also an anisotropic contribution. This is in contrast to other DFTB methods, where only a monopole approximation is done. Hence, there is only an isotropic description in other methods.

$$E^{(2)}[\rho_0, (\delta\rho)^2] \approx E^{(2)}_{\text{ES}} + E^{(2)}_{\text{XC}} + E^{(2)}_{\text{disp}} \tag{3.2.21}$$

For the third order perturbation, only the electrostatic and exchange correlation energies are used and computed with the Mulliken approximation. All other contributions to the third order perturbation are neglected.

$$E^{(3)}[\rho_0, (\delta\rho)^3] \approx E^{(3)}_{\text{ES}} + E^{(3)}_{\text{XC}} \tag{3.2.22}$$

Merging all energy contributions shown in Eq. (3.2.19) to Eq. (3.2.22) into Eq. (3.2.17) leads to Eq. (3.2.23). Though, some further adjustments are done. The isotropic contribution of the electrostatic and exchange correlation energies are described by a single term $E_{\text{IES+IEC}}$. While the anisotropic electrostatic energy E_{AES} and anisotropic exchange correlation energy E_{AXC} are described separately. The valence electron energy contributions are described via the extended Hückel type energy E_{EHT}. Independent of the above discussed perturbation, an energy term G_{Fermi} describing the Fermi–smearing is included, which leads to the complete energy equation shown in Eq. (3.2.23).

$$E_{\mathrm{GFN2-xTB}} = E_{\mathrm{rep}} + E_{\mathrm{EHT}} + E_{\mathrm{disp}} + E_{\mathrm{IES+IEC}} + E_{\mathrm{AES}} + E_{\mathrm{AXC}} + G_{\mathrm{Fermi}} \tag{3.2.23}$$

The wave function is approximated by a minimal valence basis set STO–mG. The energy is computed similar to KS–DFT and Hartree Fock. The Roothaan–Hall type Eq. (3.2.15) is solved in a selfconsistent manner.

3.3 Hessian Matrices

The hessian matrices are the target for the ML model. Therefore, a brief introduction to the hessian elements and its projection and mass weighting is given.
The hessian matrix is a $3M \times 3M$ matrix of second derivatives of the energy in respect to every permuted atom coordinate pair in the system.

$$\mathbf{H} = \begin{pmatrix} \frac{\partial^2 E}{\partial x_A \partial x_A} & \cdots & \frac{\partial^2 E}{\partial x_A \partial y_M} & \frac{\partial^2 E}{\partial x_A \partial z_M} \\ \frac{\partial^2 E}{\partial x_A \partial y_A} & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ \frac{\partial^2 E}{\partial z_N \partial x_A} & \cdots & & \frac{\partial^2 E}{\partial z_M \partial z_M} \end{pmatrix} \tag{3.3.1}$$

$\mathbf{H}$ can also be described by M^2 3×3 block matrices, where each 3×3 matrix depends on the coordinates of an atom pair. Theses 3×3 block matrices are beneficial for the prediction, as they only depend on two atoms. Hence, only information about the two atoms need to be given to the machine learning model to predict the block matrix.

$$\mathbf{H} = \begin{pmatrix} \mathbf{H}_{AA} & \mathbf{H}_{AB} & \cdots & \mathbf{H}_{AM} \\ \mathbf{H}_{BA} & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ \mathbf{H}_{MA} & \cdots\cdots & \cdots & \mathbf{H}_{MM} \end{pmatrix} \tag{3.3.2}$$

Due to the Schwartz Theorem, the $\mathbf{H}_{BA}$ and $\mathbf{H}_{AB}$ are related to another by Eq. (3.3.3)

$$\mathbf{H}_{BA} = (\mathbf{H}_{AB})^{\intercal} \tag{3.3.3}$$

Therefore, the only information needed for the hessian is a triangular matrix (s. Eq. (3.3.4)).

$$\mathbf{H} = \begin{pmatrix} \mathbf{H}_{AA} & \mathbf{H}_{AB} & \dots & \mathbf{H}_{AM} \\ & \ddots & & \vdots \\ & & \ddots & \vdots \\ & 0 & & \mathbf{H}_{MM} \end{pmatrix} \tag{3.3.4}$$

The hessian matrix needs to be projected for the computation of physical quantities like the harmonic frequencies, zero point vibrational energy. For the projection the rotational and translational parts of the hessian need to be detected. A $6 \times 3N$ matrix is constructed, that describes the rotation and translation of each atom. For one atom, the overlap is depicted in Eq. (3.3.5). The translation correspond to the first 3 rows of the matrix $\mathbf{O}_A$. While the contribution of the rotation is given in the lower 3 rows. For example, the rotation around the x–axis leads to the translation of the y–coordinate $\mathbf{r}_A^y$ by $\mathbf{r}_A^z$. While the z–coordinate is moved by $-\mathbf{r}_A^y$.

$$\mathbf{O}_A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & \mathbf{r}_A^z & -\mathbf{r}_A^y \\ -\mathbf{r}_A^z & 0 & \mathbf{r}_A^x \\ \mathbf{r}_A^y & -\mathbf{r}_A^x & 0 \end{pmatrix} \tag{3.3.5}$$

For a system of N atoms, the full matrix $\mathbf{O}$ is shown in Eq. (3.3.6)

$$\mathbf{O} = \begin{pmatrix} \mathbf{O}_A & \mathbf{O}_B & \dots & \mathbf{O}_N \end{pmatrix} \tag{3.3.6}$$

After that, the rotation and translation matrix is normalized to $\mathbf{O}_\mathrm{n}$. For the detection, the overlap matrix $\mathbf{M}$ between the normalized rotation and translation matrix and the hessian matrix is computed (s. Eq. (3.3.7)). The multiplication of the $6 \times 3N$ and $3N \times 3N$ matrix leads to $6 \times 3N$ matrix.

$$\mathbf{M} = \mathbf{O}_\mathrm{n} \times \mathbf{H} \tag{3.3.7}$$

The columns of $\mathbf{M}$ are summed up, which leads to a $3N$ vector. The maximum values of this vector corresponds to the highest overlap between a row in the Hessian matrix and the translations and rotation matrix. The six highest values are searched, and the numbers of the corresponding rows are stored in a vector $\mathbf{v}$.

For a linear molecule only the five highest values are searched. The differentiation between linear and non–linear molecules is done by the norm of the coordinates $\|\mathbf{r}^x\|$,$\|\mathbf{r}^y\|$ and $\|\mathbf{r}^z\|$. If the sum of the norm of two coordinates is below a threshold of $1 \cdot 10^{-6}$ a linear molecule is assumed. For the projection itself, the eigenvectors $\mathbf{Q}$ of the Hessian matrix and its eigenvalues $\boldsymbol{\lambda}$ are required.

$$\mathbf{H}^{\text{proj}} = \mathbf{H} - \sum_{i \in \mathbf{v}} \boldsymbol{\lambda}_i \cdot \mathbf{Q}^{\intercal}{}_i \otimes (\mathbf{Q}^{\intercal}{}_i)^{\intercal} \tag{3.3.8}$$

At last, the hessian needs to be mass weighted.

$$\mathbf{H}_{AB}^{\text{proj,w}} = \frac{1}{\sqrt{m_A m_B}} \mathbf{H}_{AB}^{\text{proj}} \tag{3.3.9}$$

Eq. (3.3.9) is applied to every hessian matrix block $\mathbf{H}_{AB}$ in the full Hessian matrix $\mathbf{H}$, which leads to the projected mass weighted Hessian matrix $\mathbf{H}^{\text{proj,w}}$.

3.4 Machine Learning

Extra Tree Regression (ETR) and Random Forest Regression (RFR) are used as ML algorithms in this research. Both algorithms are ensemble methods based on decision trees as individual estimators. In this section, decision tree based algorithms will be discussed, and the differences between the ETR and RFR will be emphasized.
Often these decision tree based algorithms are based on the Classification and Regression Tree (CART) algorithm. The CART algorithm will be explained with an example. Assuming a response Y that depends on two variables $X1$ and $X2$ is assumed. The variables $X1$ and $X2$ are called features. A tree based model splits the space of this function into several recursive binary partitions. Recursive binary means in this case, that the splits are orthogonal to the axis of the features. This split is shown on the left in Fig. 3.2.

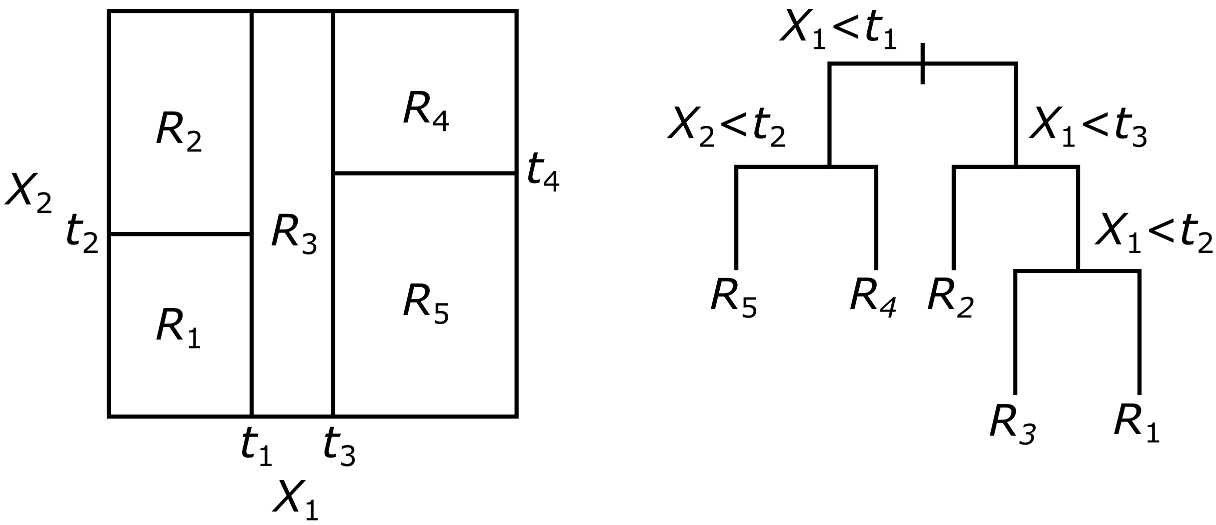

Figure 3.2: The recursively binary grown split is shown on the left. On the right, the resulting Decision Tree is depicted.

In each partition Y is modeled with a function or constant R_i. This can also be depicted in a more familiar approach, which is the decision tree. This is shown in Fig. 3.2. Both depictions lead to the same result.

The splitting points t_1 to t_4 are chosen differently for the RFR and ETR. In RFR, the splitting point is determined by choosing the best feature and of this feature the best split value. Hence, it is a two dimensional search. In the case of the ETR, the feature is chosen randomly. After that, the best split value for this feature is chosen. The relation between the features and the partitioned response Y can be depicted in a decision tree, which is shown in Fig. 3.2 on the right. This can also be described mathematically via Eq. (3.4.1)

$$\hat{f}(X) = \sum_{m=1}^{5} c_m I\{(X_1, X_2) \in R_m\} \tag{3.4.1}$$

Here, $\hat{f}(X)$ is the function that describes the response Y. Each region m has a corresponding constant c_m. The function I can either be $I = 1$, if the features X_1 and X_2 are in the rectangle R_m, or it is $I = 0$, if it is not the case. Originally, the ETR and RFR have another difference, which is that the ETR did not use bootstrapping. Bootstrapping uses the training set and makes several new training sets from this. The new training sets are randomly filled with the datasets of the original training set.

4 Methods

4.1 Feature Overview

In this section an overview of the used features is given. The quantities used for the feature vector are depicted in Tab. 4.1 and are generated with the GFN2–xTB method. The features are mostly atom centered quantities and are based on geometry, density or energy. The features were calculated for the systems in the APF CS.
There are quantities, which also have an extended version. If this is the case, a tick is set at Extension. The quadrupole moment matrix $\boldsymbol{\Theta}$ was reduced to a vector by the multiplication with a one vector $\underline{1}$ (s. Eq (4.1.1)).

$$\boldsymbol{\theta} = \boldsymbol{\Theta} \times \underline{1} \tag{4.1.1}$$

Table 4.1: List of quantities used for the ML models as features. The quantities are divided into geometry, density and energy based quantities.

Feature	Description	Extension
Geometry		
CN	Coordination Number	✓
Density		
$\boldsymbol{\mu}$	Dipole Moment	✓
$\boldsymbol{\mu}_Z$	Dipole Moment, only Z direction	
$\boldsymbol{\theta}$	Quadrupole Moment	✓
Energy		
E_{resp}	Response	
E_{gap}	HOMO – LUMO gap	✓
μ_{pot}	Chemical Potential	✓
ϵ_{HOAO}	Atomic HOMO	✓
ϵ_{LUAO}	Atomic LUMO	✓
E_{rep}	Repulsion	
E_{EHT}	extended Hückel theory	
$E_{\text{disp}}^{(2)}$	Dispersion 2$^{\text{nd}}$ order	
$E_{\text{disp}}^{(3)}$	Dispersion 3$^{\text{rd}}$ order	
$E_{\text{IES+IXC}}$	Isotropic ES and XC	
E_{AES}	Anisotropic ES	
E_{AXC}	Anisotropic XC	
E_{tot}	Total Energy	

Each quantity is atom specific, thus each quantity is there from atom A and B. The quantities shown in Tab. 4.1 are all further processed for the use as features. From the quantities x_A and x_B the arithmetic mean, the product and absolute difference is calculated (s. Eq. (4.1.2)). The new quantities X_{Arith}, X_{Prod} and X_{AbsDiff} are used as the features for the ML model.

$$X_{\text{Arith}} = \frac{x_A + x_B}{2} \tag{4.1.2a}$$

$$X_{\text{Prod}} = x_A \cdot x_B \tag{4.1.2b}$$

$$X_{\text{AbsDiff}} = |x_A - x_B| \tag{4.1.2c}$$

So far the feature X is build up by $X = \{X_{\text{Arith}}, X_{\text{Prod}}, X_{\text{AbsDiff}}\}$. If such feature is connected to the hessian matrices, it is independent for every element in a hessian matrix block $\mathbf{H}_{AB}$. Hence, the ML model is not able to distinguish the elements. Two ways were developed and tested, which distinguish the elements. At first, the feature is increased by indices specific for each element. The indices are shown in Tab. 4.2

Table 4.2: Indices used for the different elements in a hessian matrix block to differ between the elements.

Hessian element	Indexed x^{idx}	Indexed y^{idx}	Indexed z^{idx}	Permuted Index
$\frac{\partial^2 E}{\partial x_A \partial x_A}$	2	0	0	0
$\frac{\partial^2 E}{\partial x_A \partial y_A}$	1	-1	0	1
$\frac{\partial^2 E}{\partial x_A \partial z_A}$	1	0	-1	2
$\frac{\partial^2 E}{\partial y_A \partial x_A}$	-1	1	0	1
$\frac{\partial^2 E}{\partial y_A \partial y_A}$	0	2	0	0
$\frac{\partial^2 E}{\partial y_A \partial z_A}$	0	1	-1	2
$\frac{\partial^2 E}{\partial z_A \partial x_A}$	-1	0	1	2
$\frac{\partial^2 E}{\partial z_A \partial y_A}$	0	-1	1	2
$\frac{\partial^2 E}{\partial z_A \partial z_A}$	0	0	2	3

The feature vector is expanded with these indices $X = \{x^{\text{idx}}, y^{\text{idx}}, z^{\text{idx}}, X_{\text{Arith}}, X_{\text{Prod}}, X_{\text{AbsDiff}}\}$. This labeling is called **indexed** (idx).

In another approach, the hessian matrix is further rotated, so that the hessian elements are on the same spot for every prediction. In the same way, the feature quantities are adapted. The adapted matrix and coordinate system are shown in the Appendix

Table 4.3: Permutation of the coordinates for each Hessian element.

Hessian Element	x	y	z
$\frac{\partial^2 E}{\partial x_A \partial x_A}$	x	y	z
$\frac{\partial^2 E}{\partial x_A \partial y_A}$	y	x	−z
$\frac{\partial^2 E}{\partial x_A \partial z_A}$	z	x	y
$\frac{\partial^2 E}{\partial y_A \partial x_A}$	x	y	z
$\frac{\partial^2 E}{\partial y_A \partial y_A}$	y	z	x
$\frac{\partial^2 E}{\partial y_A \partial z_A}$	z	y	−x
$\frac{\partial^2 E}{\partial z_A \partial x_A}$	x	z	−y
$\frac{\partial^2 E}{\partial z_A \partial y_A}$	y	z	x
$\frac{\partial^2 E}{\partial z_A \partial z_A}$	z	x	y

After that, an index is put onto the features again. In this case though, the hessian matrix elements with a derivative in respect to the z coordinate get the same index. Furthermore, the diagonal and off–diagonal elements get different indices. This labeling is called **permuted** (perm).

4.2 Transformation of Coordinates

The hessian matrix and some features depend on the orientation of the coordinate system (CS). For the ML model a consistent orientation for every atom pair is necessary. Furthermore, the hessian elements shall be learned atom pair wise. Thus, the orientation of the molecule is brought into atom pair focused CS, where the origin of the CS is in the center of the considered atom pair, while both atoms are on the z–Axis. In this section, the transformation of the initial coordinate system into the atom pair focused CS will be discussed.

The initial CS is not fixed to any constraints. Therefore, the initial CS is transformed into the main inertia system, from which the transformation to the atom pair focused coordinate systems is performed.

At first a translation of the origin of the initial CS to the center of mass is done by Eq. (4.2.1).

$$\mathbf{r}_A^{\mathrm{s}} = \mathbf{r}_A^{\mathrm{init}} - \mathbf{s}^{\mathrm{init}} \tag{4.2.1}$$

Where $\mathbf{r}_A^{\mathrm{s}}$ is the coordinate vector of an atom A in the mass centered CS, $\mathbf{r}_A^{\mathrm{init}}$ the coordinate vector in the initial CS and $\mathbf{s}_A^{\mathrm{init}}$ is the center of mass in the initial CS, and is defined in Eq. (4.2.2).

$$\mathbf{s}^{\mathrm{init}} = \frac{1}{M}\sum_{A}^{N} m_A \cdot \mathbf{r}_A^{\mathrm{init}} \tag{4.2.2}$$

Here m_A is the mass of atom A, M is the mass of the molecule and N the number of atoms in the molecule. Once the origin of the new CS is determined, the orientation of its axes is arranged to the moment of inertia as computed by Eq. (4.2.3).

$$\mathbf{I}^{\mathrm{s}} = \begin{pmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{pmatrix}; I_{ab} = \begin{cases} \sum_A^N m_A(b_A^2 + c_A^2) & \text{if } a = b \\ -\sum_A^N m_A \cdot a_A \cdot b_A & \text{if } a \neq b \end{cases} \qquad a, b \in x, y, z \tag{4.2.3}$$

For the computation of the elements I_{ab} of the inertia tensor $\mathbf{I}^{\mathrm{s}}$ the diagonal elements and non–diagonal elements are divided in two cases. Here, a and b are variables for the spatial directions of the coordinates x, y and z. For the rotation of the mass centered origin CS into the main inertia system, the matrix $\mathbf{R}_{\mathrm{mi}}^{\mathrm{init}}$ of eigenvectors of the inertia tensor $\mathbf{I}^{\mathrm{s}}$ is needed. For that, the eigenvectors $\mathbf{R}_{\mathrm{mi}}^{\mathrm{init}}$ are computed by solving the eigenvalue equation. The eigenvalues are then used to get the eigenvectors. To assure, that the matrix $\mathbf{R}_{\mathrm{mi}}^{\mathrm{init}}$ is a right–handed matrix the determinant is calculated. If the determinant is negative, the sign of the third eigenvector is changed. Furthermore, it needs to be assured, that the eigenvector is consistently rotated in the same direction, independent of the outgoing position. The chosen constraint is, that the highest value of the eigenvectors are positive, if this is not the case the sign of the eigenvector and the next eigenvector in $\mathbf{R}_{\mathrm{mi}}^{\mathrm{init}}$ is changed.
The rotation of the coordinates is done by Eq. (4.2.4).

$$\mathbf{r}_A^{\mathrm{mi}} = \mathbf{R}_{\mathrm{mi}}^{\mathrm{init}} \mathbf{r}_A^{\mathrm{s}} \tag{4.2.4}$$

Here, $\mathbf{r}_A^{\mathrm{mi}}$ is the coordinate of an atom A in the main inertia CS. The rotation is done for every atom in the molecule. After that operation, the molecule is in the main inertia CS.
The hessian matrix also depends on the CS and therefore needs to be rotated, too. A translation is not needed. Eq. (4.2.6) describes the rotation of the hessian matrix in the initial CS to the main inertia system. For this a $3N \times 3N$ rotation matrix $\mathbf{P}_{\mathrm{mi}}^{\mathrm{init}}$ needs to be constructed, where

the diagonal 3 x 3 blocks are $\mathbf{R}_{\text{mi}}^{\text{init}}$. This matrix is shown in Eq. (4.2.5).

$$\mathbf{P}_{\text{mi}}^{\text{init}} = \begin{pmatrix} \mathbf{R}_{\text{mi}}^{\text{init}} & \mathbf{0} & & \mathbf{0} \\ \mathbf{0} & \mathbf{R}_{\text{mi}}^{\text{init}} & & \mathbf{0} \\ \mathbf{0} & & \ddots & \\ & & \ddots & \mathbf{0} \\ \mathbf{0} & & \mathbf{0} & \mathbf{R}_{\text{mi}}^{\text{init}} \end{pmatrix} \tag{4.2.5}$$

$$\mathbf{H}^{\text{mi}} = \mathbf{P}_{\text{mi}}^{\text{init}} \times \mathbf{H}^{\text{init}} \times \left(\mathbf{P}_{\text{mi}}^{\text{init}}\right)^{\mathsf{T}} \tag{4.2.6}$$

The hessian matrix in the initial system is $\mathbf{H}^{\text{init}}$, while the hessian matrix in the main inertia system is $\mathbf{H}^{\text{mi}}$.

In the following, the transformation of the main inertia CS to the atom pair focused CS will be discussed. The transformation starts with the translation of the origin to the center of an atom pair. The translation is described with Eq. (4.2.7).

$$\mathbf{r}_{C}^{\text{mi,c}} = \mathbf{r}_{C}^{\text{mi}} - \left(\frac{1}{2}\mathbf{r}_{A}^{\text{mi}} + \frac{1}{2}\mathbf{r}_{B}^{\text{mi}}\right) \text{ for } A, B, C \in A, B, ..., N \tag{4.2.7}$$

After this operation, the origin of the CS is in the center of the atom pair. To fix the orientation, the atom pair is aligned along the z–axis. The Euler angles are used to determine the rotation matrix. The rotation of the CS depends on the considered atom pair. It is divided into whether $A = B$ or $A \neq B$. At first the construction of the rotation matrix for the case $A \neq B$ will be discussed.

The z–axis of the atom pair focused CS is aligned to $\mathbf{r}_{A}^{\text{mi,c}}$. The x–axis is defined in Eq. (4.2.8), where $\mathbf{z}$ is a unit vector in z–direction of the atom pair focused CS, and $\boldsymbol{\mu}_A$ is the dipole moment of the atom A.

$$\mathbf{x} = \mathbf{z} \times (\boldsymbol{\mu}_{\text{A}} + \boldsymbol{\mu}_{\text{B}}) \tag{4.2.8}$$

The cutting line $\mathbf{L}$ of the xy–plane of the atom pair focused CS and xy–plane of the centered main inertia CS is described with Eq. (4.2.9).

$$\mathbf{L} = \mathbf{x} \times \mathbf{z}^{\text{mi,c}} \tag{4.2.9}$$

The cutting line $\mathbf{L}$, the axes $\mathbf{x}, \mathbf{z}, \mathbf{x}^{\text{mi,c}}$ and $\mathbf{z}^{\text{mi,c}}$ are then used for the computation of the Euler angles α, β and γ via Eq. (4.2.10).

$$\alpha^0 = \arccos\left(\frac{\mathbf{L} \cdot \mathbf{x}^{\mathrm{mi,c}}}{|\mathbf{L}| \cdot |\mathbf{x}^{\mathrm{mi,c}}|}\right) \tag{4.2.10a}$$

$$\beta = \arccos\left(\frac{\mathbf{x} \cdot \mathbf{z}^{\mathrm{mi,c}}}{|\mathbf{x}| \cdot |\mathbf{z}^{\mathrm{mi,c}}|}\right) \tag{4.2.10b}$$

$$\gamma^0 = \arccos\left(\frac{\mathbf{L} \cdot \mathbf{x}}{|\mathbf{L}| \cdot |\mathbf{x}|}\right) \tag{4.2.10c}$$

For the right rotation, it has to be considered, whether the vectors $\mathbf{x}^{\mathrm{mi,c}}, \mathbf{z}^{\mathrm{mi,c}}$ and $\mathbf{L}$ span a right–handed coordinate system or left–handed. This is done by constructing a matrix of these vectors and computing the determinant. The determinant can be used, as it is equal to the triple product for a 3×3 matrix.

$$\mathbf{A} = (\mathbf{x}, \mathbf{z}, \mathbf{L}) \tag{4.2.11}$$

$$\mathbf{G} = (\mathbf{L}, \mathbf{x}^{\mathrm{mi,c}}, \mathbf{z}^{\mathrm{mi,c}}) \tag{4.2.12}$$

$$\alpha = \begin{cases} \alpha^0 & \text{if } \det(\mathbf{A}) = 1 \\ 2\pi - \alpha^0 & \text{if } \det(\mathbf{A}) = -1 \end{cases} \tag{4.2.13}$$

$$\gamma = \begin{cases} \gamma^0 & \text{if } \det(\mathbf{G}) = -1 \\ 2\pi - \gamma^0 & \text{if } \det(\mathbf{G}) = 1 \end{cases} \tag{4.2.14}$$

Via these constraints, the Euler angles α and γ are chosen. The Euler angles are then used to build up the rotation matrices via Eq. (3.1.1) and (3.1.2).
The complete rotation matrix is then constructed by the cross product of the single rotation matrices and is shown in Eq. (4.2.15).

$$\mathbf{R}^0 = \mathbf{R}_z(\gamma) \times \mathbf{R}_x(\beta) \times \mathbf{R}_z(\alpha) \tag{4.2.15}$$

If the x–values of the rotated dipole moment $\boldsymbol{\mu}$ are negative, the matrix is rotated by 180° around the z–axis. This is depicted in Eq. (4.2.16).

$$\mathbf{R}_{\mathrm{mi}}^{\mathrm{APF}} = \begin{cases} \mathbf{R}^0 & \text{if } \boldsymbol{\mu}^x > 0 \\ \mathbf{R}^0 \times \mathbf{R}_z(\pi) & \text{if } \boldsymbol{\mu}^x < 0 \end{cases} \tag{4.2.16}$$

Lastly, the rotation of the translated coordinate system is done via Eq. (4.2.17).

$$\mathbf{r}_A^{\mathrm{APF}} = \mathbf{R}_{\mathrm{mi}}^{\mathrm{APF}} \times \mathbf{r}_A^{\mathrm{mi,c}} \tag{4.2.17}$$

The coordinates $\mathbf{r}_A^{\mathrm{APF}}$ are the final coordinates in the APF CS for the case of a heteronuclear atom pair.

For the case of a homonuclear atom pair, the procedure is the same up to the translation of the coordinates into the centered inertial main CS. The z–axes of APF CS and centered inertial main CS are aligned. Therefore, the Euler angle β is zero. Furthermore, the xy–planes are the same, and there is no cutting line of the xy–planes. Thus, only one angle α or γ is necessary for the rotation. Therefore, γ is the angle between the x–axis of the centered main inertial CS and the APF CS. To provide consistency in the rotation of the coordinates, the x–axis is chosen in dependency of the dipole moment $\boldsymbol{\mu}$ of the considered atom. Eq. (4.2.18) is show how the x–axis is computed.

$$\mathbf{x} = \mathbf{z} \times \boldsymbol{\mu}_A \tag{4.2.18}$$

This results in the computation of the angle γ^0 by Eq. (4.2.19).

$$\gamma^0 = \arccos\left(\frac{\mathbf{x}^{\mathrm{mi,c}} \cdot \mathbf{x}}{|\mathbf{x}^{\mathrm{mi,c}}| \cdot |\mathbf{x}|}\right) \tag{4.2.19}$$

Between two vectors are two angles, which sum up to 2π. The rotation of the CS depends on the arrangement to another. The angle is chosen by the following condition, show in Eq. (4.2.20).

$$\gamma = \begin{cases} \gamma^0 & \text{if } \mathbf{x}_y < 0 \\ 2\pi - \gamma^0 & \text{if } \mathbf{x}_y \geq 0 \end{cases} \tag{4.2.20}$$

The further procedure of the transformation is done in the same way as for the heteronuclear atomic pair. The rotation matrix is computed via Eq. (4.2.15). This followed by an adjustment of the rotation matrix, shown in Eq. (4.2.16). The final rotation matrix $\mathbf{R}_{\mathrm{mi}}^{\mathrm{APF}}$ is then used on the coordinates $\mathbf{r}_A^{\mathrm{mi,c}}$ via Eq. (4.2.17).

In the APF CS the computation of the hessian was done. For each APF CS one 3×3 hessian matrix block is of interest. To rebuild the hessian matrix in the mi system, each hessian matrix block needs to undergo a rotation back to the mi system. For this the all rotation matrices $\mathbf{R}_{\mathrm{mi}}^{\mathrm{APF}}$ need to be stored. The rotation back into the mi system is done by Eq. (4.2.21).

$$\mathbf{H}_{\mathrm{AB}}^{\mathrm{mi}} = \left(\mathbf{R}_{\mathrm{mi}}^{\mathrm{APF}}\right)^{\mathsf{T}} \times \mathbf{H}_{\mathrm{AB}}^{\mathrm{APF}} \times \mathbf{R}_{\mathrm{mi}}^{\mathrm{APF}} \tag{4.2.21}$$

In the mi system, the hessian matrix $\mathbf{H}^{\mathrm{mi}}$ is filled with the blocks $\mathbf{H}^{\mathrm{mi}}_{\mathrm{AB}}$. Before some quantities for measuring can be computed, the $\mathbf{H}^{\mathrm{mi}}$ is projected and mass–weighted. Thus, the $\mathbf{H}^{\mathrm{proj,w|mi}}$ is build. The procedure of the projection and mass–weighting is described in the chapter Theory.

4.3 Quantities for Measuring

The mean squared error MSE was chosen for the quantification of the learning process of the ML models. The ML models are trained with GFN2–xTB based information. Therefore, the reference is also GFN2–xTB. In Eq. (4.3.1) is the MSE shown. Here, y corresponds to the hessian elements, calculated with GFN2–xTB and predicted with the ML model. The number of data points is n.

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_{i,\mathrm{ML}} - y_{i,\mathrm{xTB}}\right)^2 \tag{4.3.1}$$

For the comparison of the different models (e.g. ETR–Case–Perm, RFR–Gen–Idx, etc.) the comparison is done by assuming a normal distribution.
For this the difference between the GFN2–xTB computation and ML model prediction is calculated. Furthermore, the standard deviation and mean is computed. After that a normal distribution is assumed. This normal distribution is computed with Eq. (4.3.2).

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}}\exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right) \tag{4.3.2}$$

The physical quantities, that were investigated are the elements of the hessian matrix $\mathbf{H}^{\mathrm{mi}}$, the wave number $\tilde{\nu}$, the force constant k and the zero point vibrational energy ZPVE. The wave numbers are calculated by Eq. (4.3.3a), the force constants by Eq. (4.3.3b) and the ZPVE by Eq. (4.3.3c).
Here, λ are the eigenvalues of the mass weighted projected hessian matrix $\mathbf{H}^{\mathrm{proj,w|mi}}$ and m_i the mass of each atom. For the transformation into SI–units, the hartree Energy E_{h}, planck constant h, reduced planck constant $\hbar$, speed of light c and the atomic mass unit u are required.

$$\tilde{\nu} = \sqrt{|\lambda|}\cdot\frac{E_{\mathrm{h}}}{\hbar\cdot c} \tag{4.3.3a}$$

$$k = \frac{E_{\mathrm{h}}^2}{\hbar^2\cdot u}\cdot\sum_{i}^{M}\frac{1}{m_i}\lambda \tag{4.3.3b}$$

$$\mathrm{ZPVE} = \frac{E_{\mathrm{h}}}{h}\sum_{i}^{n}\sqrt{|\lambda_i|} \tag{4.3.3c}$$

4.4 Computational Methods

All electronic structure computations were performed with GFN2–xTB[1], for this *xtb*[2] was used. The computation of the information for the feature vector and target vector are done with GFN2–xTB level calculation. For this the GFN2–xTB code was used. For transformation of the coordinates and the ML models python was used. Especially, the packages *sklearn*[3], *numpy*[4] and *pandas*[5,6] were used for this project.

5 Results and Discussion

5.1 The ML Models and Data Set

For every prediction of the hessian matrix, it is necessary to use the homonuclear and heteronuclear model. Both models were further investigated by three adjustments, namely the algorithms, the labeling and a generalization.

At first, as described in the Chapter Theory, the RFR and ETR are used as the different algorithms for the fitting of the ML model. Secondly, there are two possibility for the labeling. Either the indexed (Idx) case is used or the permuted (Perm) case. The third adjustment is the distinction between a case specific (Case) and a general (Gen) model. For the case specific models, only one molecular system with different structures is used for the training of a model. While in the general model all molecular systems are used to train one single model. For the further discussion, abbreviations are used for the description of the different variations of the model. As an example, the abbreviation for the model with the ETR as the algorithm, the indexed labeling and general case is ETR–Gen–Idx.

A data set for benchmarking the ML model was constructed. The set consists of eight different molecules. These are namely H_2, N_2, O_2, F_2, HF, CO, H_2O, and NH_3. For each molecule, structures with different bond length were generated. Only the distance of one bond was changed for each molecule. This lead into 103 different structures. These structures were split into a training set of 75% and a test set of 25%. The split is done structure wise to achieve the possibility to compare other quantities then the hessian elements. For each structure, the hessian and feature were calculated with the GFN2–xTB method.

Moreover, the performance of the model to a complete unknown system is of interest. Therefore, a H_3–system was used for this which is predicted by the model. The full data set named above was used as a training set for the model, while the H_3–system was used as the test set.

5.2 The H_2 system

As a first approach, the ETR–Case–Perm model was used for the subset of the hydrogen molecule. Here, the wave numbers $\tilde{\nu}_{\mathrm{ML}}$ were predicted and plotted against the via GFN2–xTB calculated wave numbers $\tilde{\nu}_{\mathrm{xTB}}$. This is done for 5 random states and is shown in Fig. 5.1. The black line is the diagonal, on which the data points would appear for a perfect prediction. Furthermore, the wave number increases with the decrease of the bond length. For the approximated range of $1800\ \mathrm{cm}^{-1} < \tilde{\nu}_{\mathrm{xTB}} < 7500\ \mathrm{cm}^{-1}$ there are abbreviations from the diagonal and therefore, in respect to the small data set of eleven structures, a sufficient prediction.

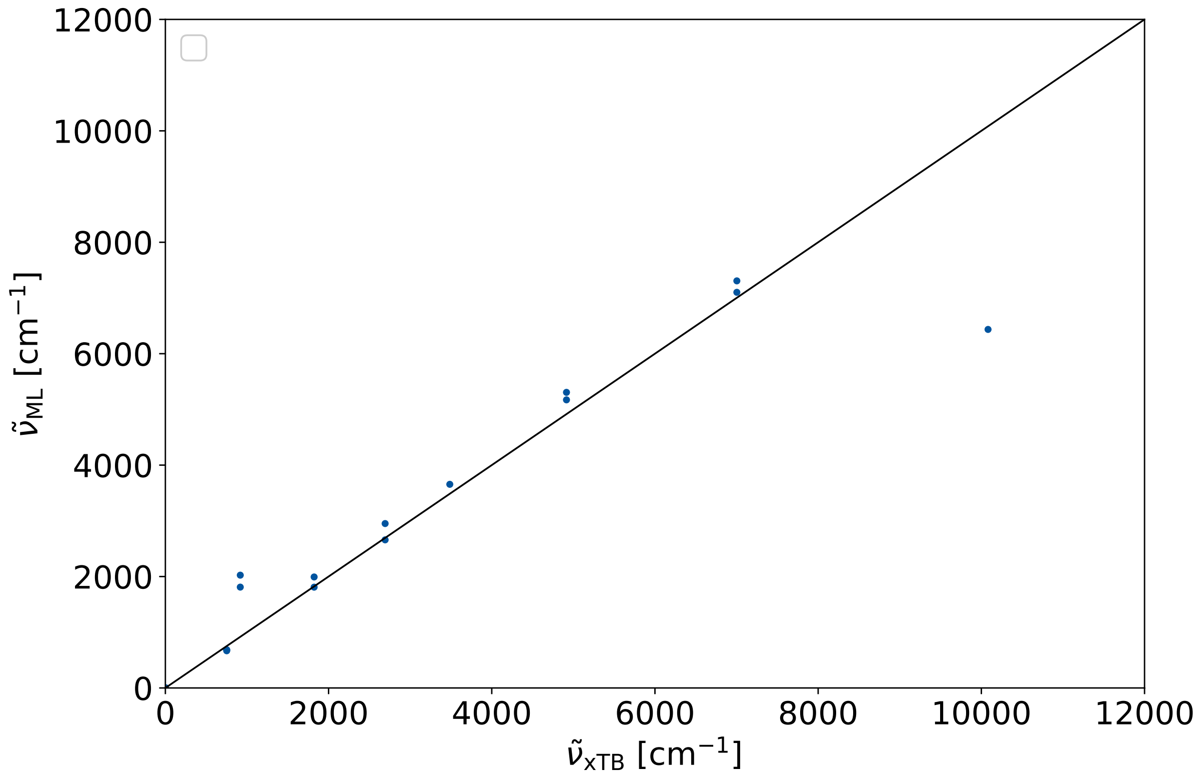

Figure 5.1: The wavenumber, predicted via the ML model $\tilde{\nu}_{\mathrm{ML}}$ is plotted against the wavenumber calculated via GFN2–xTB $\tilde{\nu}_{\mathrm{xTB}}$. The model uses the Extra Tree Regression. The sample set is H_2 and the training set is 75 % of the whole set. The prediction were done for 5 random states.

The abbreviations are stronger in the cases of $1800\ \mathrm{cm}^{-1} > \tilde{\nu}_{\mathrm{xTB}}$ and $\tilde{\nu}_{\mathrm{xTB}} > 7500\ \mathrm{cm}^{-1}$. These predictions are probably done for cases, where the training set does not include molecules, which have a lower or higher distance, respectively. Hence, it can be assumed, that the error is due to extrapolation of the ML model. This assumption is underlined by the fact, that the predicted values correspond to the next higher or lower value. For example, the next lowest value in the case of the prediction at $\tilde{\nu}_{\mathrm{xTB}} \approx 1 \cdot 10^5 \mathrm{cm}^{-1}$ is $\tilde{\nu}_{\mathrm{xTB}} \approx 7 \cdot 10^4 \mathrm{cm}^{-1}$. This in the same order as the predicted value $\tilde{\nu}_{\mathrm{ML}} \approx 6.4 \cdot 10^4 \mathrm{cm}^{-1}$ at $\tilde{\nu}_{\mathrm{xTB}} \approx 1 \cdot 10^5 \mathrm{cm}^{-1}$. This matches to the expected behavior for a decision tree based algorithms (cf. Fig. 3.2).

This concludes in the sufficient description of the wave numbers for interpolation. It could also be shown, that the prediction fails for extrapolation.

5.3 Comparison of RFR and ETR

For the comparison of the algorithms, the RFR–Gen-Perm and ETR–Gen–Perm models were used. These models are compared to each other by testing there dependency on the amount of training set used. As the quantity for testing the performance the MSE between the predicted hessian and the hessian computed via GFN2–xTB is used. The MSE was calculated for ten different random states and the mean and standard deviation of these MSE was calculated.
It is expected, that the performance of the ML model should increase, the more data it is trained with. This can be interpreted as a statistical learning behavior. To verify this behavior, the training set was increased successively. The full training set is 75 % of the full data set. Thus, the test set is 25 % of the data set. While keeping the test set constant, the training set is changed successively between 7.5 % of the data set to 75 %. Fig 5.2 shows the MSE in the dependency of the amount of training set used for the ETR (on the left) and the RFR (on the right). The error bars indicate the standard deviation.

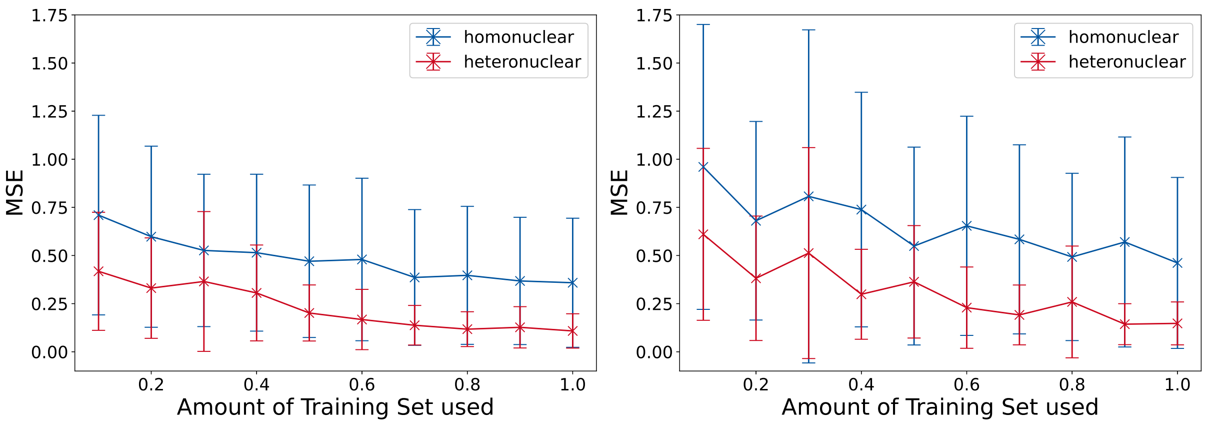

Figure 5.2: Learning plots for the predicting hessian elements $\frac{\partial^2 E}{\partial a \partial b}$ for an increase of the amount of training set used. On the left is the learning plot of the Extra Tree Regression and on the right the plot of the Random Forest Regression shown. The full training set is 75 % of the sample set. The full sample set, excluding the H_3 system, is used. The mean of ten random states were used for the MSE. The plots are done for the homonuclear model (blue) and heteronuclear model (red).

In the case of the ETR, the MSE decreases with an increase of the amount of training set used for the homonuclear and heteronuclear model. This behavior is also recognizable in the case of the RFR. In comparison to the ETR, the RFR shows a higher standard deviation for the data points. Thus, the RFR depends more strongly on the random state than the ETR.

Furthermore, the ETR has, especially for the homonuclear model, lower MSE values for any amount of training set. At the full amount of training set and at 90% of the training set, the RFR and ETR show a similar performance for the heteronuclear model. All in all, the ETR shows a better performance for the whole investigated range.
There is the possibility, that the ML model has so much information in the features, that it can always predict the target. Hence, the model is tested on whether this is the case, or the ML model learns from the information of the feature. For this, the features are scattered with a random factor. The intensity of the scattering is amplified by the factor σ. It is expected, that a model that learns from the features performs worse, if the scattering of the data points increases.

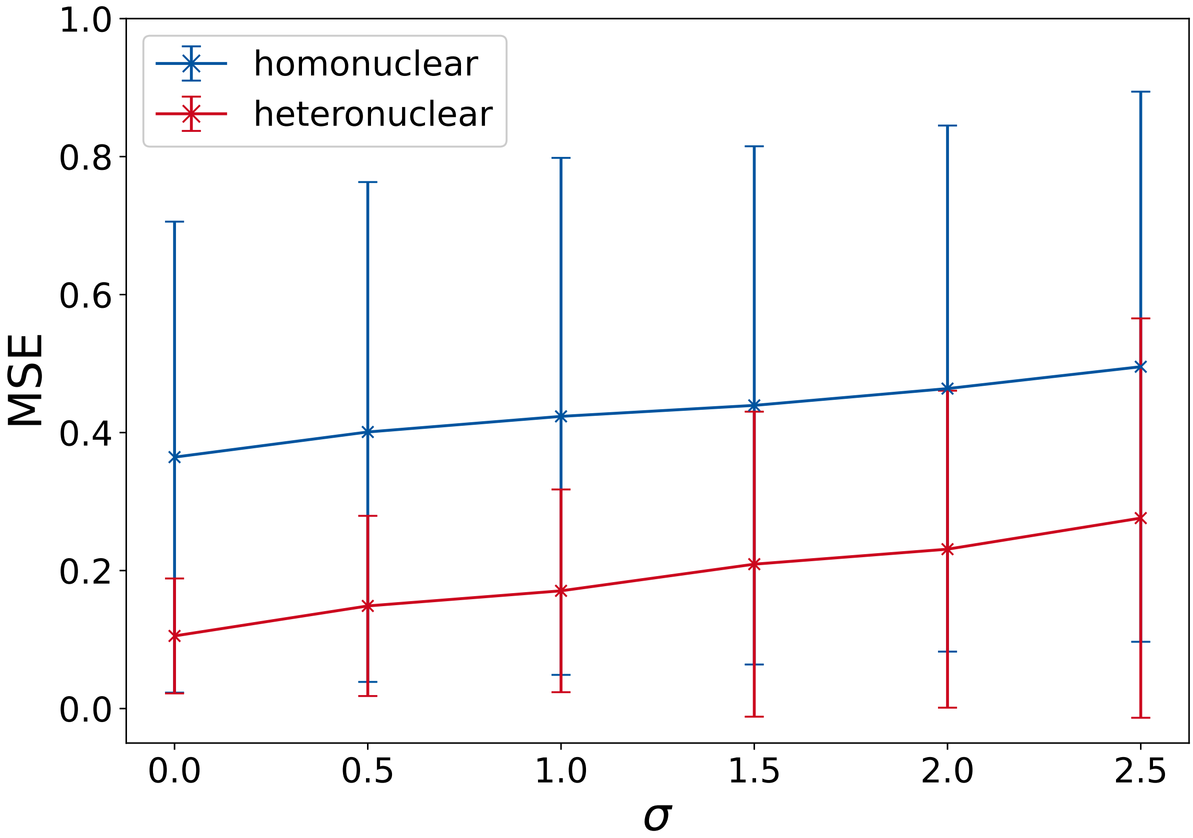

Figure 5.3: Learning plots for predicting hessian elements $\frac{\partial^2 E}{\partial a \partial b}$ for a variation of the factor σ. The full sample set, excluding the H_3 system, is used for the Ml models. The mean of ten random states were used for the MSE. The plots are done for the homonuclear model (blue) and heteronuclear model (red).

The performance of the model ETR–Multi–Perm was tested for this. The prediction was done for ten different random states. The MSE between the predicted hessian elements and the via GFN2–xTB calculated ones was investigated. For the investigated range of $0 \leq \sigma \leq 2.5$ the MSE increases monotonically. Hence, this shows, that the model learns from the features and is not purely random.

In this section a comparison of the RFR and ETR was done. The ETR shows overall a better performance than the RFR. Furthermore, it could be verified, that the ML model learns statistically from the features used for the prediction of the hessian matrices.

5.4 Comparison of Permutation Model and Indexed Model

Beside the different algorithms for the construction of the trees, the models with the variations of Gen/Case and Idx/Perm were compared. Here, the models ETR–Gen–Perm, ETR–Gen–Idx, ETR–Case–Perm and ETR–Case–Idx are investigated. The quantities for measuring are the hessian elements and the wave numbers of the harmonic vibration. The difference between the predicted and via GFN2–xTB computed quantities were computed. From these differences the standard deviation as also the mean were calculated, and a Gaussian distribution was assumed. At first the direct prediction of the hessian elements were investigated, and its distribution is shown in Fig 5.4.

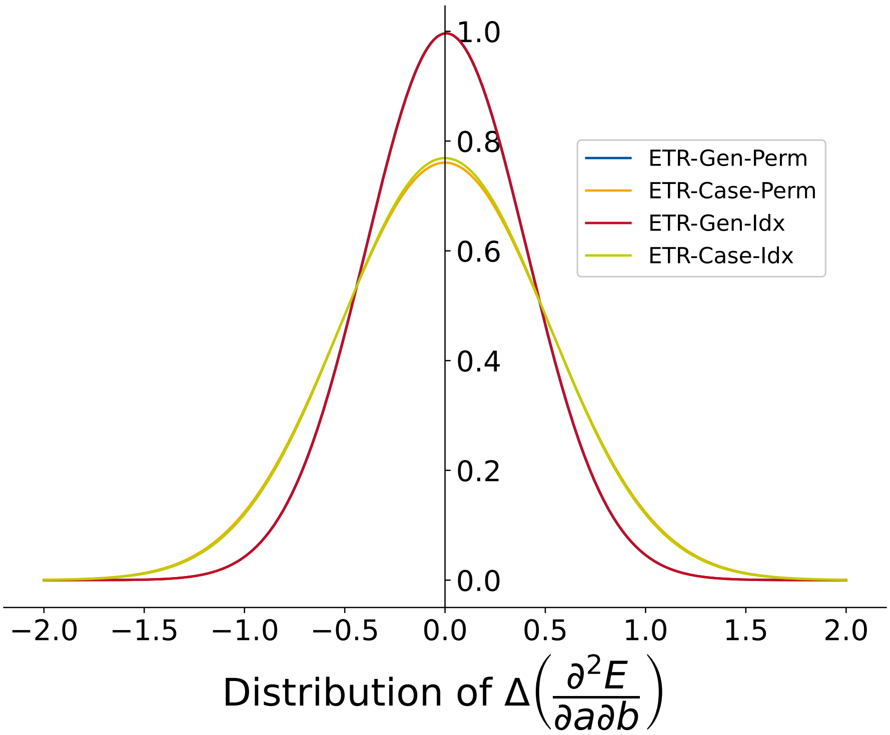

Figure 5.4: Distribution of $\Delta\left(\frac{\partial^2 E}{\partial a \partial b}\right)$ for different models. In every model, the Extra Tree Regression was used. The distribution is assumed to be Gaussian type. For this the median and standard deviation of $\Delta\left(\frac{\partial^2 E}{\partial a \partial b}\right)$ are computed.

Comparing the ETR–Case–Perm and ETR–Case–Idx, as also ETR–Gen–Perm and ETR–Gen–Idx shows, that both approaches for the labeling of the features do not differ significantly. Hence, both approaches are in good agreement with another. This shows, on the one hand, the strength of the decision tree algorithms.

They cannot overfit and seem to be capable of describing this complex system with a simple differentiation by indexes. On the other hand, the permuted model does the same prediction for the model, but may be useful for other machine learning algorithms or even neural networks. The comparison of the ETR–Case–Perm and ETR–Gen–Perm shows, that the ETR–Case–Perm model deviates more strongly. This is in agreement with the comparison of the ETR–Case–Idx and ETR–Gen–Idx models. Though the deviation is stronger, the means of the models do not differ significantly. The data set for training the models is smaller for the case–specific models than for the general models. This might be the reason for the deviation.

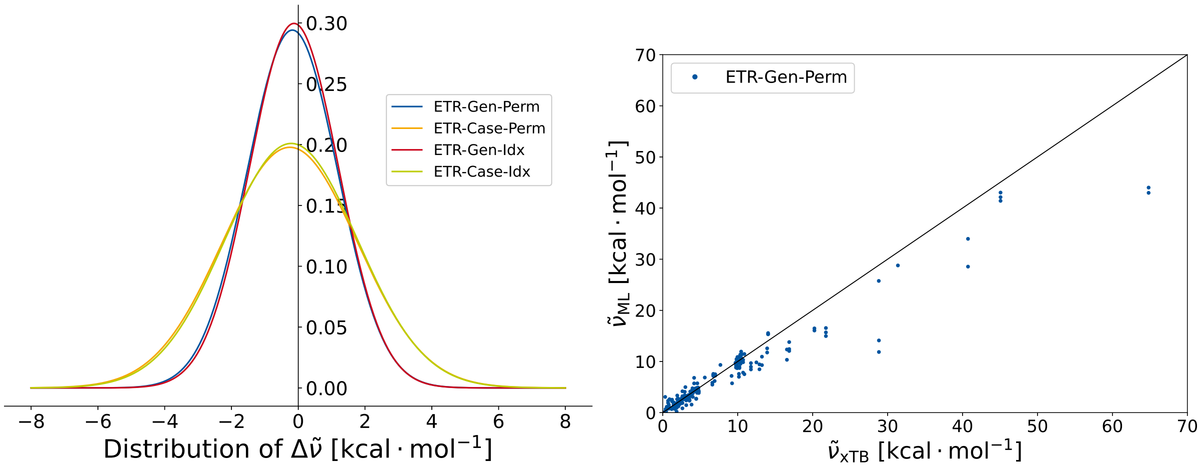

Figure 5.5: On the left is the distribution of $\Delta\tilde{\nu}$ for different models. In every model, the Extra Tree Regression was used. The distribution is assumed to be Gaussian type. For this the median and standard deviation of $\Delta\tilde{\nu}$ are computed. On the right is the $\tilde{\nu}_{\mathrm{ML}}$ plotted against $\tilde{\nu}_{\mathrm{xTB}}$. The wave numbers are predicted with the ETR–Gen–Perm model.

The wave numbers $\tilde{\nu}$ of the harmonic frequencies of the molecules were calculated. The difference between the difference between the ML predicted $\tilde{\nu}_{\mathrm{ML}}$ and the via GFN2–xTB computed $\tilde{\nu}_{\mathrm{xTB}}$ was calculated. A Gaussian distribution was assumed. This is plotted in Fig. 5.5 for the four different varieties of the model. Furthermore, the plot of the $\tilde{\nu}_{\mathrm{ML}}$ vs. $\tilde{\nu}_{\mathrm{xTB}}$ is shown. The relative distributions of the four variations of the models are similar to the ones of the hessian elements. The ETR–Gen–Perm and ETR–Gen–Idx show a better performance than the ETR–Case–Perm and the ETR–Case–Idx. The mean is shifted to a negative $\Delta\tilde{\nu}$. In contrast to the Gaussian distribution of the hessian elements, the Idx models show a marginal better performance than the Perm model. The deviations of several $\mathrm{kcal} \cdot \mathrm{mol}^{-1}$ are rather strong. These strong deviations might be due to high extrapolation errors, which lead to high standard distribution. For example at $\tilde{\nu}_{\mathrm{xTB}} \approx 65\mathrm{kcal} \cdot \mathrm{mol}^{-1}$ the deviation might be due to extrapolation. Hence, the difference of about $\Delta\tilde{\nu} \approx 20\mathrm{kcal} \cdot \mathrm{mol}^{-1}$ has a strong influence on the distribution, although ML is not customized for extrapolation problems.

Contrary to the extrapolation errors are the wave numbers, which correspond to rotation and translation. These wave numbers are approximately zero and have large contribution to number of wave numbers. Therefore, the narrow the standard deviation, and shift the mean to zero. Further investigations are necessary, to see whether the extrapolation or the wave numbers, corresponding to rotation and translation, have a higher impact onto the mean and standard deviation.

5.5 Benchmarking the H_3 system

Especially, the prediction of hessian matrices of fully unknown systems is of interest. Hence, the ETR-Gen-Perm and ETR-Gen-Idx models were tested on the performance on a new, for the model unknown, system. For the H_3 system the outer hydrogen atoms were fixed, while the inner hydrogen atom was shift from one hydrogen to another. For each position of the hydrogen atom, the force constants of the systems were predict via the ML model. The force constants were also calculated via GFN2–xTB.

Fig. 5.6 shows the force constants calculated by GFN2–xTB and predicted via the ETR–Gen–Idx model. Interesting to point out is the symmetric behavior of the system. The ML model should, in principle, mimic this behavior.

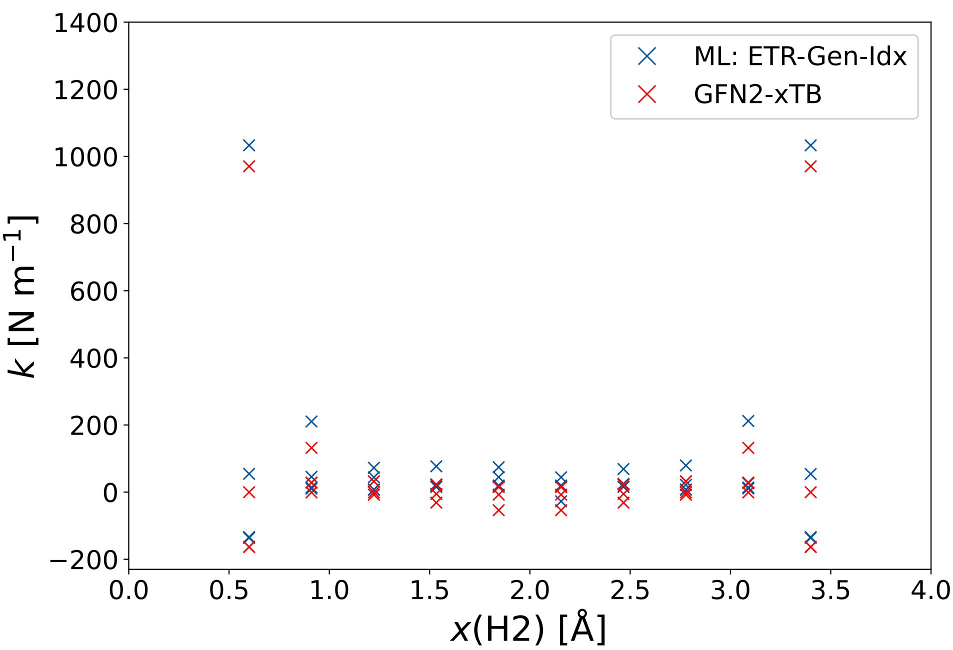

Figure 5.6: Force constants predicted for the H_3 systems in dependence of the distance between H1 and H2 are shown. The force constants are shown for the GFN2–xTB calculation (red), as also the via the ML model predicted values (blue). The full sample set, excluding the H_3 system, was used for the ML model. The ML model ETR–Gen–Idx was used for the prediction.

While this is overall the case, there are abbreviation from this behavior. For the example, the symmetry is not given in the case of $x(\mathrm{H2}) = 1.84$ Å and $x(\mathrm{H2}) = 2.15$ Å.
Fig. 5.7 shows the prediction of the focre constants via the ETR–Gen–Perm model. Similar to the ETR–Gen–Idx model, the symmetry is in general achieved. Nevertheless, for example the force constants at $x(\mathrm{H2}) = 0.91$ Å and $x(\mathrm{H2}) = 3.08$ Å show abbreviations from oneanother.

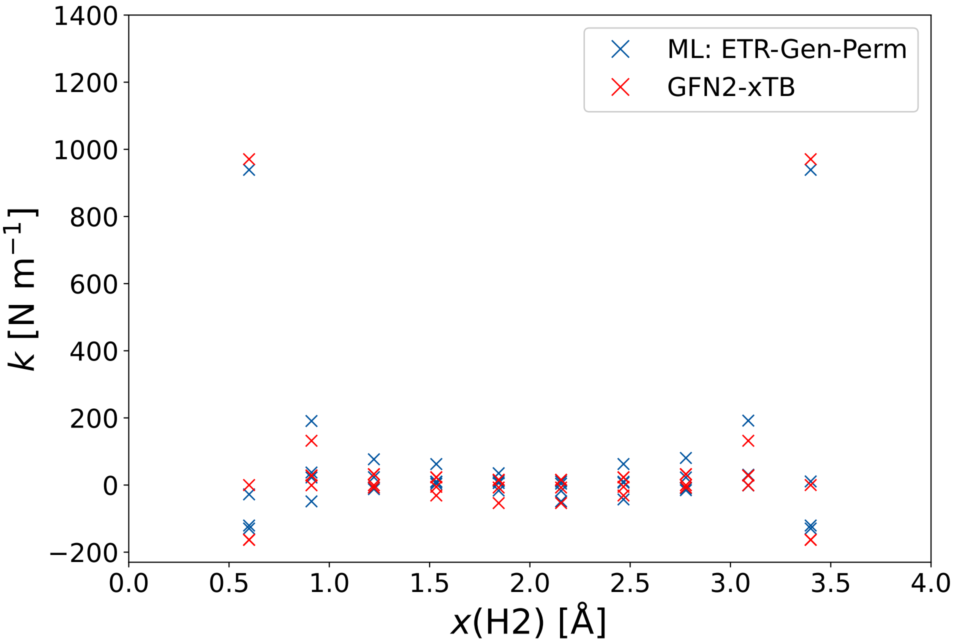

Figure 5.7: Force constants predicted for the H_3 systems in dependence of the distance between H1 and H2 are shown. The force constants are shown for the GFN2–xTB calculation (red), as also the via the ML model predicted values (blue). The full sample set, excluding the H_3 system, was used for the ML model. The ML model ETR–Gen–Perm was used for the prediction.

An overall sufficient prediction of the force constants was achieved, though there are still errors in the symmetry. Further investigations on the models are necessary, to locate the reason for the break of symmetry. Worth considering are the rotation process, the homonuclear and heteronuclear model. The rotation process can be neglected, as rotation is also done for the GFN2–xTB calculations. A decoupled investigation of the homonuclear and heteronuclear model is therefore necessary.

6 Conclusion & Outlook

The goal of the research was the prediction of hessian matrices by a machine learning model. The machine learning model was based on atomic features and the prediction of the elements of the hessian matrices was done atom pair wise. The model was benchmarked by a small data set of 8 molecules with 103 different structures. The computations of the hessian matrices and features for these systems was done with the GFN2–xTB method.
In first place, the general learning of the model was investigated. A statistical learning behavior of the ML models RFR–Gen–Perm and ETR–Gen–Perm could be observed. Compared to another, the ETR–Gen–Perm model showed a better performance. Furthermore, a physical correlation could be presented for the ETR–Gen–Perm model.
Moreover, 4 variations of the model were investigated and compared to another, which are the models ETR–Gen/Case–Perm/Idx. Here a similar behavior for different labeling approaches (Indexed and Permuted) was shown. The general models show a better prediction than the case–specific models. Here, the hessian elements, zero point vibrational energy and wave numbers were predicted.
In addition, the set was used for the prediction of a system, that is unknown to the ML model. The data set of the new H_3 system shows a symmetric behavior for the force constants, which could, with some minor errors, be depicted by the ML models.

The next necessary approaches for this research, is the improvement of the models in there symmetric behavior. Although, for the H_3 system the errors in symmetry are minor, the adjustment to a full symmetric behavior is mandatory. In addition, a set of 103 structures is small for a ML model. Hence, the date set size should be increased and tests on benchmark data sets should be done for a comparison to literature. When the prediction of the hessian matrix on GFN2–xTB level is sufficient, the model can be adapted to a ΔML model.

7 Appendix

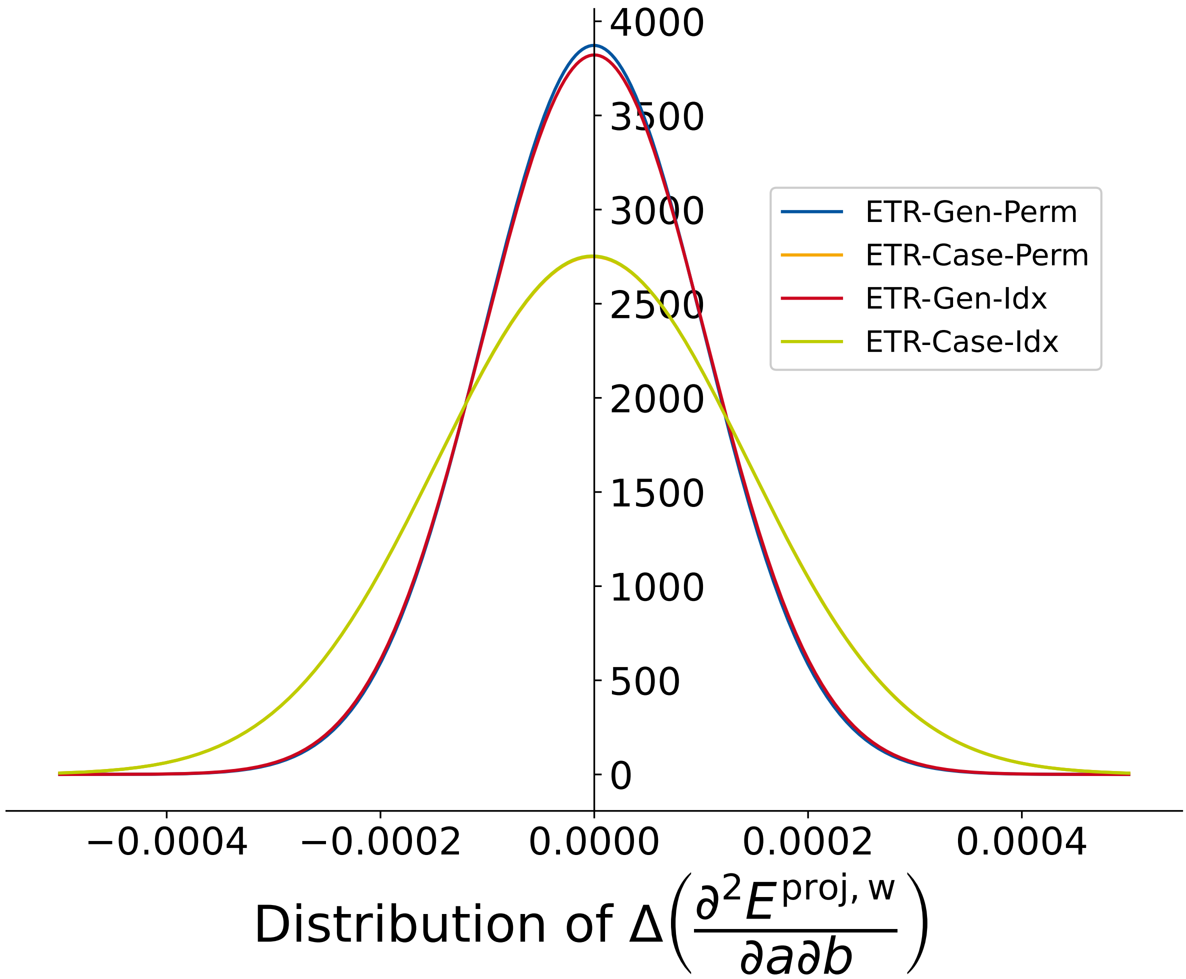

Figure 7.1: Distribution of $\Delta\left(\frac{\partial^2 E^{\text{proj,w}}}{\partial a \partial b}\right)$ for different models. In every model, the Extra Tree Regression was used. The distribution is assumed to be Gaussian type. For this the median and standard deviation of $\Delta\left(\frac{\partial^2 E^{\text{proj,w}}}{\partial a \partial b}\right)$ are computed.

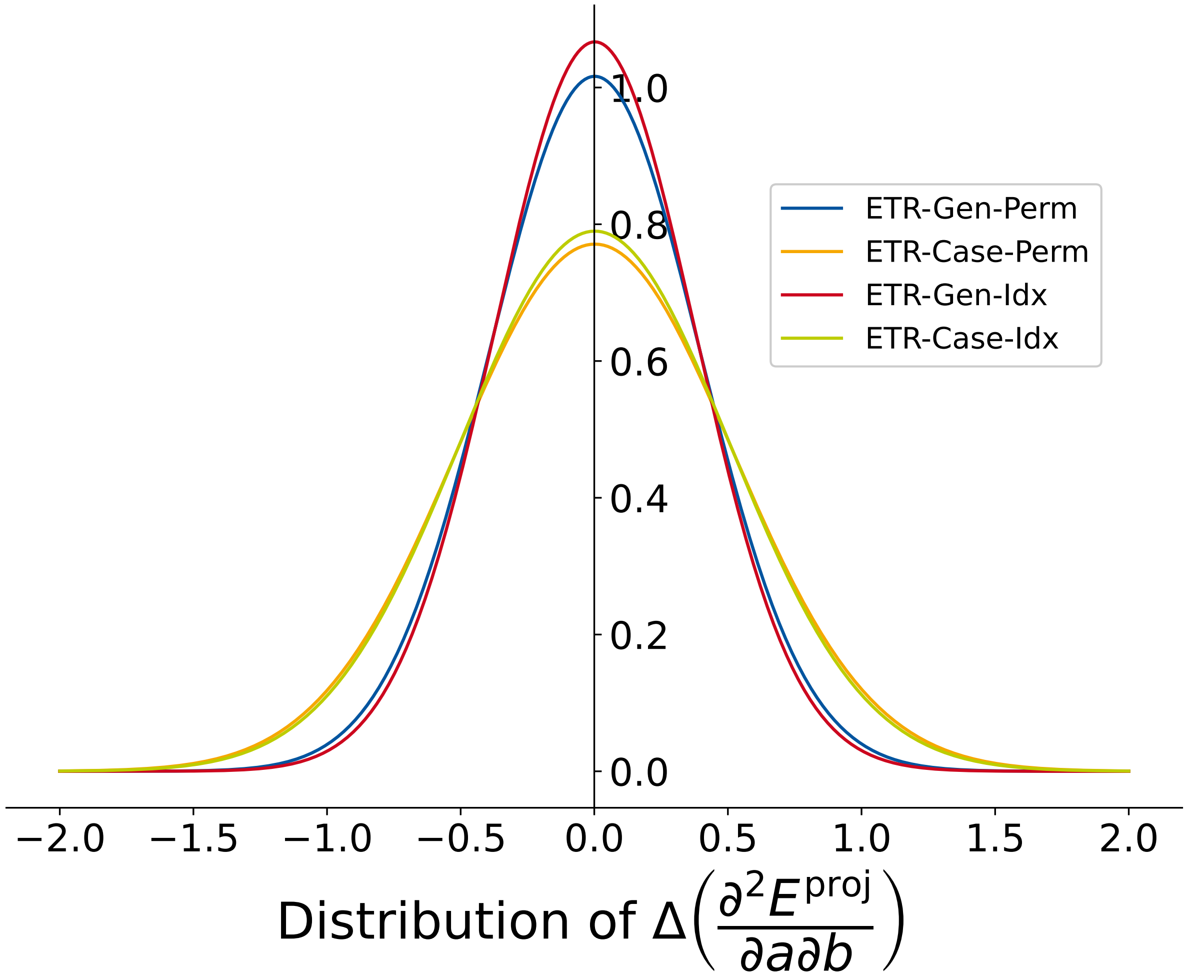

Figure 7.2: Distribution of $\Delta\left(\frac{\partial^2 E^{\text{proj}}}{\partial a \partial b}\right)$ for different models. In every model, the Extra Tree Regression was used. The distribution is assumed to be Gaussian type. For this the median and standard deviation of $\Delta\left(\frac{\partial^2 E^{\text{proj}}}{\partial a \partial b}\right)$ are computed.

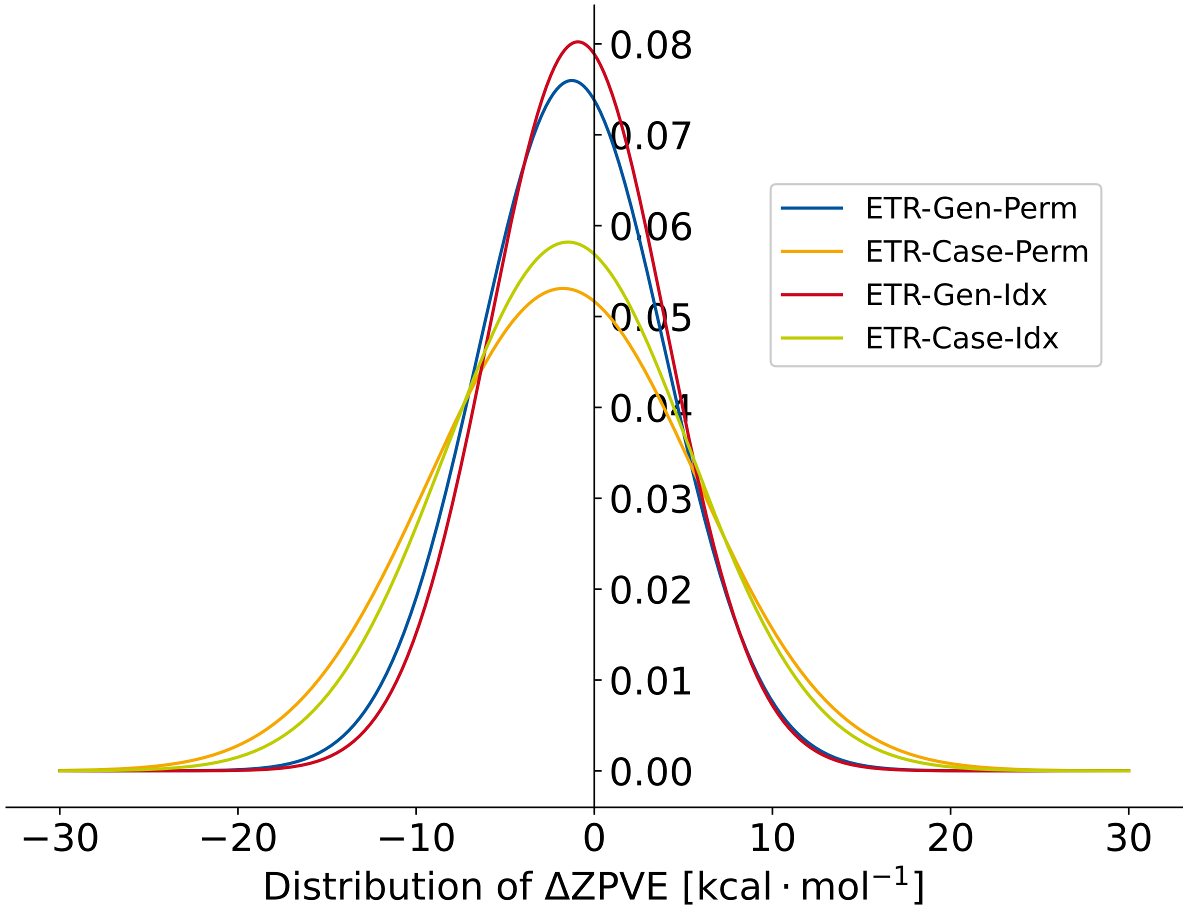

Figure 7.3: Distribution of ΔZPE for different models. In every model, the Extra Tree Regression was used. The distribution is assumed to be Gaussian type. For this the median and standard deviation of ΔZPE are computed.

$$CN_A = \sum_{B \neq A}^{N_{\text{atoms}}} \frac{1}{1 + \exp\left(\frac{-16a \cdot 4(R_{A,\text{cov}} + R_{B,\text{cov}})}{3R_{AB} - 1}\right)} \tag{7.0.1a}$$

$$CN^{\text{ext}} = \sum_{B \neq A}^{N_{\text{atoms}}} \frac{CN_B}{f_{\text{damp}}(R_A, R_B)} \tag{7.0.1b}$$

$$q_A = Zg_A - \sum_{\mu \in A}^{N_{\text{BF}}} \sum_{\nu}^{N_{\text{BF}}} P_{\mu\nu} S_{\mu\nu} \tag{7.0.1c}$$

$$q_A^{\text{ext}} = q_A + \sum_{B \neq A}^{N_{\text{atoms}}} \frac{q_B}{f_{\text{damp}}} \tag{7.0.1d}$$

$$E_{A,\text{EHT}} = \sum_{k \in A} \sum_{k} P_{k\lambda} H_{k\lambda} \tag{7.0.1e}$$

$$E_{A,\text{rep}} = \sum_{B \neq A}^{M} \frac{1}{2} \frac{Y_A^{\text{eff}} Y_B^{\text{eff}}}{R_{AB}} \cdot \exp\left(\alpha_A \alpha_B\right)^{0.5} R_{AB}^{k_{\text{rep}}} \tag{7.0.1f}$$

$$E_{A,\text{disp}}^{(2)} = \sum_{A \neq B} \sum_{n=6,8} -s_n \frac{1}{2} \frac{C_n^{AB}(q_a, CN_{\text{cov}}^A, q_b, CN_{\text{cov}}^B)}{R_{AB}^n} f_n^{\text{damp,BJ}}(R_{AB}) \tag{7.0.1g}$$

$$\begin{aligned} E_{A,\text{disp}}^{(3)} = {} & \sum_{B \neq A} \sum_{C \neq B \neq A} -s_9 \frac{1}{3} \frac{(3\cos(\theta_{ABC})\cos(\theta_{BCA}\cos(\theta_{CBA}) + 1))}{(R_{AB} R_{AC} R_{BC})^3} \\ & \cdot C_9^{ABC}(CN_{\text{cov}}^A, CN_{\text{cov}}^B, CN_{\text{cov}}^C) \cdot f_9^{\text{damp,zero}}(R_{AB}, R_{AC}, R_{BC}) \end{aligned} \tag{7.0.1h}$$

$$\begin{aligned} E_{A,\text{AES}} &= E_{A,q\mu} + E_{A,q\theta} + E_{A,\mu\mu} \\ &= \sum_{B \neq A} \frac{1}{2} \Big(f_3(R_{AB}) \left[q_A(\boldsymbol{\mu}_B^{\intercal} \mathbf{R}_{BA}) + q_B(\boldsymbol{\mu}_A^{\intercal}) \mathbf{R}_{AB} \right] \\ &+ f_5(R_{AB}) \left[q_A \boldsymbol{R}_{BA}^{\intercal} \boldsymbol{\Theta}_B \boldsymbol{R}_{BA} + q_B \boldsymbol{R}_{BA}^{\intercal} \boldsymbol{\Theta}_A \boldsymbol{R}_{BA} \right. \\ &- \left. 3(\boldsymbol{\mu}_A^{\intercal} \boldsymbol{R}_{AB})(\boldsymbol{\mu}_B^{\intercal} \boldsymbol{R}_{AB}) + (\boldsymbol{\mu}_A^{\intercal} \boldsymbol{\mu}_B R_{AB}^2) \right] \Big) \end{aligned} \tag{7.0.1i}$$

$$E_{A,\text{AXS}} = \left(f_{\text{XC}^{\mu_A} |\boldsymbol{\mu}_A|^2} + f_{\text{XC}^{\Theta_A} ||\boldsymbol{\Theta}_A||^2} \right) \tag{7.0.1j}$$

$$E_{A,\text{IES+IXC}} = \frac{1}{2} \sum_{B \neq A} \sum_{l \in A} \sum_{l' \in B} q_{A,l} q_{B,l'} \gamma_{AB,ll'} + \frac{1}{3} \sum_{l \in A} \Gamma_{A,l} q_{A,l}^3 \tag{7.0.1k}$$

8 Bibliography

[1] C. Bannwarth, S. Ehlert, and S. Grimme, "GFN2-xTB—an accurate and broadly parametrized self-consistent tight-binding quantum chemical method with multipole electrostatics and density-dependent dispersion contributions," *Journal of Chemical Theory and Computation*, vol. 15, no. 3, pp. 1652–1671, Feb. 2019. DOI: `10.1021/acs.jctc.8b01176`. [Online]. Available: `https://doi.org/10.1021/acs.jctc.8b01176`.

[2] C. Bannwarth *et al.*, "Extended tight-binding quantum chemistry methods," *WIREs Computational Molecular Science*, vol. 11, no. 2, e1493, 2021. DOI: `https://doi.org/10.1002/wcms.1493`. eprint: `https://wires.onlinelibrary.wiley.com/doi/pdf/10.1002/wcms.1493`. [Online]. Available: `https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/wcms.1493`.

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. DOI: `10.1038/s41586-020-2649-2`. [Online]. Available: `https://doi.org/10.1038/s41586-020-2649-2`.

[5] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., 2010, pp. 56–61. DOI: `10.25080/Majora-92bf1922-00a`.

[6] T. pandas development team, *Pandas-dev/pandas: Pandas*, version latest, Feb. 2020. DOI: `10.5281/zenodo.3509134`. [Online]. Available: `https://doi.org/10.5281/zenodo.3509134`.